
Arquitectura de despliegue

Portal institucional

Agencia ITRC

Documento técnico de referencia

Dirigido a equipo de infraestructura y administradores del sitio

Contenido del documento:

- Migración de WordPress a JAMstack (Astro 4.x estático)
- Capa CMS dockerizada (Strapi v5 + Postgres) como fuente del contenido
- Despliegue automatizado con *runner* self-hosted en el servidor
- Arquitectura sobre servidor institucional (Ubuntu + nginx)
- Flujo de trabajo combinado: edición visual (Strapi) + Git (código)
- Manejo separado de contenido (Postgres) y binarios (FS del servidor)
- Política de respaldos local (snapshots con *hardlinks*)

Índice

1. Resumen ejecutivo	2
1.1. Componentes principales	3
2. Diagrama de arquitectura	4
3. Flujo de publicación paso a paso	5
3.1. Escenario 1: Webmaster edita en Strapi (flujo cotidiano)	5
3.2. Escenario 2: Cambio de código o de schema (push desde VS Code)	5
3.3. (Histórico) Sveltia CMS — sustituido	6
3.4. Escenario 3: Subida de binarios al servidor	6
3.5. Escenario 4: <i>Deploy</i> manual de fallback	6
4. Configuración del servidor (Ubuntu + nginx)	8
4.1. Servidor de pruebas actual	8
4.2. Cuentas y permisos del servidor	8
4.3. Estructura del sitio en el servidor	8
4.4. Configuración nginx (estado actual)	8
5. Autenticación y administración de usuarios	10
5.1. Modelo de cuentas	10
5.2. Flujo de autenticación (OAuth con GitHub) — <i>pendiente de despliegue</i>	10
5.3. Alta de un webmaster nuevo	10
5.4. Roles sugeridos	11
5.5. Baja de un webmaster	11
5.6. Buenas prácticas de seguridad	11
6. Consideraciones operativas	12
6.1. Manejo de binarios	12
6.2. Rollback (revertir un cambio)	12
6.3. Monitoreo y logs	13
6.4. Política de respaldos local	13
6.5. Respaldo del repositorio (Git)	13
Referencias	14

1 Resumen ejecutivo

El portal institucional de la Agencia ITRC migró desde una arquitectura WordPress (dinámica, basada en base de datos MySQL y PHP) hacia una arquitectura **JAMstack** basada en el generador de sitios estáticos **Astro**. El sitio público sigue siendo **HTML estático servido por nginx**; la novedad de mayo 2026 es la incorporación de **Strapi v5** como capa CMS headless: el contenido editorial se administra en una base de datos Postgres y se consume vía API durante el build de Astro. El despliegue automatizado lo orquesta un *runner* self-hosted de GitHub Actions que vive en el mismo servidor.

Cambios clave respecto al modelo anterior

- **Sitio público estático:** el servidor sólo entrega HTML/CSS/JS/PDF prerenderizados; no hay PHP ni consultas dinámicas a base de datos en tiempo de respuesta al ciudadano.
- **CMS headless con base de datos propia:** Strapi v5 + Postgres corren en contenedores en el mismo servidor, accesibles para editores en `/admin/`. Astro consulta el API de Strapi en tiempo de *build*, no en cada *request*.
- **Git como fuente de verdad para código y schemas:** el código del portal y los schemas de Strapi viven en el repo (`cms-strapi/src/api`, `cms-strapi/src/components`). El contenido editorial vive en Postgres.
- **Despliegue automatizado:** tanto un `git push` a `main` como un evento `publish` en Strapi disparan el workflow que reconstruye el sitio.
- **Runner self-hosted:** la construcción ocurre dentro del propio servidor, evitando exponer SSH a la nube y eliminando llaves remotas.
- **Binarios fuera del repo y de la BD:** PDFs y multimedia residen sólo en el filesystem del servidor, con respaldo local nocturno (decisión revisada en mayo 2026).

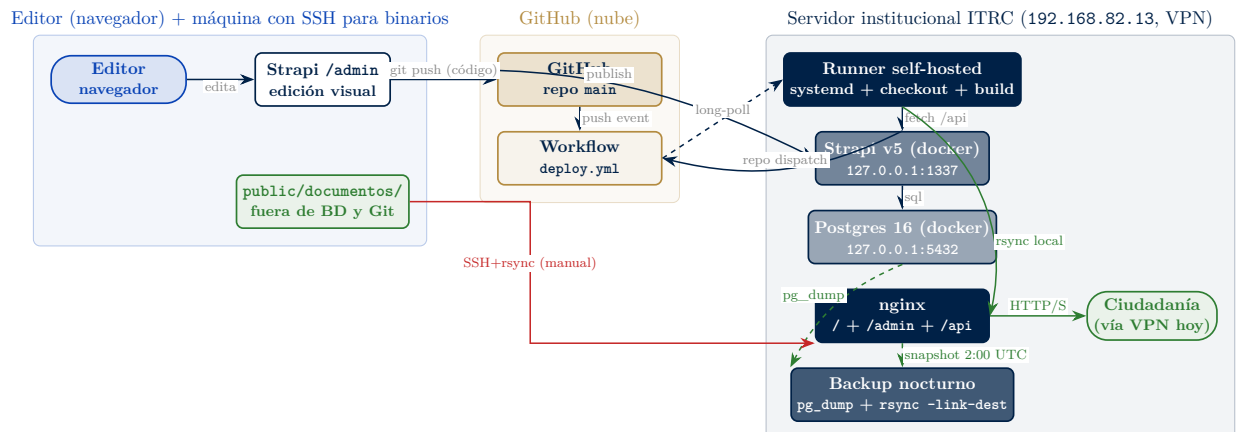
Estado del despliegue (mayo 2026)

El portal está operativo en un **servidor de pruebas** accesible únicamente desde la VPN institucional (FortiClient) en `http://192.168.82.13/`. El despliegue automatizado funciona extremo a extremo. **La migración masiva de Sveltia CMS a Strapi v5 quedó completada en mayo de 2026:** 135 single-types + 7 colecciones (318 documentos) auto-generados desde los JSON originales y servidos al build de Astro. Pendiente para producción pública: dominio definitivo, certificado TLS, refactor del menú de Strapi para reflejar la navegación del sitio, y replicación del estado al server vía script.

1.1 Componentes principales

Componente	Rol
Astro 4.x	Generador de sitio estático. Consume el API de Strapi en <i>build-time</i> y produce HTML.
Strapi v5 (CE)	CMS headless con panel administrativo en <code>/admin/</code> . 142 content types, 219 components reusables, 318 documentos.
Postgres 16	Base de datos de Strapi. Corre en Docker, expuesta sólo a <code>127.0.0.1:5432</code> .
Docker + Compose	Orquesta los contenedores Strapi y Postgres en el mismo host. Imagen Strapi construida desde <code>cms-strapi/Dockerfile</code> .
GitHub	Almacenamiento del código y schemas, control de versiones, orquestador de despliegues.
GitHub Actions	Pipeline CI/CD que construye y publica el sitio. Workflow: <code>.github/workflows/deploy.yml</code> . Acepta <code>push</code> y <code>repository_dispatch[strapi-publish]</code> .
Runner self-hosted	Servicio <code>systemd</code> ejecutado por el usuario <code>github-runner</code> dentro del servidor. Etiquetas: <code>[self-hosted, itrc-server]</code> .
Ubuntu + nginx	Servidor institucional. nginx sirve el HTML estático y proxea <code>/admin/</code> , <code>/api/</code> , <code>/uploads</code> hacia Strapi (<code>127.0.0.1:1337</code>).
rsync + cron	Política de respaldos local con snapshots tipo <i>Time Machine</i> usando <i>hardlinks</i> . <code>pg_dump</code> previo al <code>rsync</code> para Postgres.

2 Diagrama de arquitectura



Cómo leer el diagrama

Las flechas **azules continuas** representan acciones del usuario o del sistema disparadas por un **push**. Las **azules punteadas** representan el *long-poll* por el cual el runner espera trabajos de GitHub (sin abrir puertos de entrada). Las **verdes** representan el camino del contenido publicado hasta la ciudadanía y el respaldo nocturno. La flecha **roja** representa el canal manual de subida de binarios (PDFs, multimedia) que viven fuera del repositorio.

Diferencia clave con la versión anterior

La versión 1.0 corría el GitHub Action en la nube con SSH al servidor. La 1.1 lo migró a *runner* self-hosted dentro del servidor, eliminando la necesidad de exponer SSH. La **versión 1.2 (mayo 2026)** agrega Strapi v5 + Postgres como capa CMS dockerizada en el mismo servidor; el contenido editorial pasó de archivos JSON commiteados al repo (modelo Sveltia CMS) a una base de datos administrada vía panel web. Astro consulta Strapi en tiempo de *build*, así el sitio público sigue siendo HTML estático.

3 Flujo de publicación paso a paso

El portal soporta cuatro flujos de publicación. El primero (edición visual en Strapi) es el flujo cotidiano del webmaster; el segundo (push desde VS Code) cubre cambios de código y schemas; los otros dos (binarios y *deploy* manual) son canales secundarios.

3.1 Escenario 1: Webmaster edita en Strapi (flujo cotidiano)

Aplica para todo el contenido editorial: textos, secciones, listas de documentos, banners, noticias. Es el flujo principal a partir de mayo 2026.

1. El webmaster navega a `<dominio>/admin/` (proxiado por nginx hacia Strapi en `127.0.0.1:1337`).
2. Inicia sesión con su usuario administrativo de Strapi (gestionado en el propio panel, no requiere GitHub OAuth).
3. En el menú lateral elige *Content Manager* y selecciona el *single type* o *collection* a editar (p. ej. *Marco Legal*, *Normograma*).
4. Edita los campos en el formulario visual; cada *single type* expone sólo los campos del schema correspondiente.
5. Click en *Save* guarda como *draft* (no publica). Click en *Publish* crea la versión publicada.
6. Un *middleware* en el backend (`strapi.documents.use`) intercepta el evento `publish/unpublish/delete` y emite un `repository_dispatch[strapi-publish]` a GitHub vía API (token PAT con scope `Actions:write`).
7. GitHub dispara el workflow `deploy.yml`; el runner ejecuta `npm run build` con `STRAPI_URL` apuntando a `http://127.0.0.1:1337`. Astro consulta los 142 endpoints (`/api/...`) y prerenderiza el sitio.
8. `rsync` local + reload de nginx, igual que el Escenario 2.
9. Tiempo total: 2 a 4 minutos desde el click *Publish*.

3.2 Escenario 2: Cambio de código o de schema (push desde VS Code)

Aplica para cambios técnicos: layout, estilos, lógica de Astro, schemas de Strapi, scripts de migración. Requiere conocer Git.

1. El desarrollador ejecuta `git pull` en su copia local para sincronizarse.
2. Abre el proyecto en VS Code y modifica archivos: schemas en `cms-strapi/src/api/`, componentes Astro en `src/components/`, scripts, etc. Si modificó schemas, ejecuta `node cms-strapi/scripts/autogen-schema.js` y `node cms-strapi/scripts/gen-strapi-fetchers.mjs`.
3. Ejecuta `npm run dev` localmente para previsualizar en `http://localhost:4321` (con su Strapi local levantado en `localhost:1337`).
4. Satisfecho, `git commit` y `git push`.
5. GitHub recibe el push y, salvo que los archivos modificados estén en `paths-ignore` (`docs/**`, `README.md`, `CLAUDE.md`, `LICENSE`, `.gitignore`, `GEMINI.md`), dispara el workflow `deploy.yml`.
6. El *runner self-hosted* (esperando trabajos vía *long-poll*) ejecuta los pasos: `checkout`, `npm ci`, `npm run build` (con `STRAPI_URL` y `STRAPI_TOKEN` desde *repo variables*), y `rsync -a -delete --exclude=/documentos/` desde `dist/` a `/var/www/itrc-web/`.

- Si los schemas cambiaron, en el server se requiere `docker compose up -d --build strapi` para que Strapi tome el nuevo set de content types antes del build (paso manual hoy; pendiente automatizar dentro del workflow).
- El runner emite `sudo /usr/bin/systemctl reload nginx` (única regla sudoers que tiene). nginx recarga la configuración sin downtime.
- Tiempo total: 1 a 3 minutos. El estado del workflow se observa en GitHub → Actions.

Configuración del workflow

El archivo `.github/workflows/deploy.yml` usa `concurrency: deploy-${{ github.ref }}` para serializar despliegues, `timeout-minutes: 15` como salvaguarda, y un bloque `env:` para todas las interpolaciones de `${{ ... }}` dentro de pasos `run:` (mitigación de *shell injection*, identificada por *semgrep*). Acepta tanto `push` a `main` como `repository_dispatch` con `event_type=strapi-publish`.

3.3 (Histórico) Sveltia CMS — sustituido

Cerrado en mayo 2026

La versión 1.1 de este documento describía aquí Sveltia CMS como pieza pendiente: en su versión 0.128.x sólo permitía subir media a servicios pagados (Cloudinary, Uploadcare), incompatible con el presupuesto del ITRC. La decisión de mayo 2026 fue migrar a **Strapi v5 CE** (gratis, autohospedado, base de datos propia, panel administrativo completo). La migración masiva de los 318 documentos quedó hecha (ver §3.1) y los archivos JSON originales en `src/content/pages/` se conservan como respaldo de origen hasta consolidar el flujo Strapi en producción.

3.4 Escenario 3: Subida de binarios al servidor

Los archivos binarios (PDFs, imágenes pesadas, multimedia) viven **fuera del repositorio Git**, sólo en el servidor (decisión de mayo 2026, ver §6.1). Para subirlos:

- El operador edita o añade archivos localmente en `public/documentos/`.
- Ejecuta `npm run deploy:binarios`, que invoca `ops/deploy-binarios.sh`.
- El script abre una conexión SSH (usuario `admweb`, autenticado con llave) hacia `192.168.82.13` y sincroniza con `rsync -avz -no-o -no-g -no-p -no-t` (sin `-delete` por seguridad).
- Los binarios quedan disponibles en `/var/www/itrc-web/documentos/` para servirse con cache largo (`expires 7d`).

Por qué un canal manual y no el workflow

El runner haría `rsync -delete` sobre el webroot, borrando todo lo que no esté en el repo. Como los binarios no están en el repo, los borraría. La regla `-exclude=/documentos/` en el workflow protege el directorio durante despliegues automáticos y obliga a manejar binarios por un canal separado y consciente.

3.5 Escenario 4: *Deploy* manual de fallback

Si el runner está caído (mantenimiento, fallo del servicio), el operador puede desplegar manualmente desde su máquina:

- `npm run build` localmente.

2. `npm run deploy` (script `ops/deploy.sh`, lee `.env.deploy` con `SSH_BIN`, `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_PATH`).
3. El script hace `rsync` desde `dist/` hacia el servidor con las mismas exclusiones.

Este flujo no requiere que el runner esté corriendo y funciona sólo con SSH desde la máquina del operador.

Trazabilidad

Cada commit queda registrado en el historial Git con autor, fecha y descripción. Cada despliegue automático queda registrado en GitHub → Actions con SHA, autor y mensaje del commit. El *deploy* manual queda en el historial `rsync` del operador, sin trazabilidad central — por eso es un escenario de excepción, no rutinario.

4 Configuración del servidor (Ubuntu + nginx)

4.1 Servidor de pruebas actual

Atributo	Valor
Hostname / IP	192.168.82.13 (sólo accesible vía VPN FortiClient institucional)
Sistema operativo	Ubuntu Server 22.04 LTS
nginx	1.18+ (paquete oficial Ubuntu)
Espacio en disco	16 GB total, ~12 GB libres tras instalación + 2 snapshots
RAM	2 GB
CPU	1 vCPU (suficiente para servir estáticos)
Puertos abiertos	80 (HTTP, vía VPN), 22 (SSH)
TLS	pendiente (sin dominio público todavía)

4.2 Cuentas y permisos del servidor

El servidor cuenta con dos usuarios funcionales y un grupo `deploy` compartido con `setgid` para que ambos puedan escribir el mismo árbol de archivos:

Usuario	Rol
<code>admweb</code>	Operador humano. Acceso SSH con llave; recibe binarios desde la máquina local del operador (<code>npm run deploy:binarios</code>). Miembro del grupo <code>deploy</code> .
<code>github-runner</code>	Cuenta de servicio que ejecuta el runner self-hosted (vía <code>systemd</code>). Tiene una regla <code>sudoers</code> muy acotada que le permite <i>únicamente</i> <code>/usr/bin/systemctl reload nginx</code> . Miembro del grupo <code>deploy</code> .
<code>deploy</code> (grupo)	Grupo compartido. Webroot <code>/var/www/itrc-web/</code> es del grupo <code>deploy</code> con <code>g+rwX</code> y bit <code>g+s</code> (<code>setgid</code>) para que archivos nuevos hereden el grupo.

Sudoers del runner

```
github-runner ALL=(root) NOPASSWD: /usr/bin/systemctl reload nginx
```

Nada más. El runner no puede instalar paquetes, cambiar configuración, ni reiniciar servicios. Esto limita el blast radius de un compromiso del repo o del workflow.

4.3 Estructura del sitio en el servidor

```
/var/www/itrc-web/      <-- webroot (raiz para nginx)
|- index.html           <-- copiados aquí por el runner
|- _astro/              <-- assets contruidos por Astro
|- ...                  <-- resto del sitio compilado
\-- documentos/         <-- binarios (PDFs, multimedia)
                        NO los toca el runner;
                        se suben por canal separado
```

4.4 Configuración nginx (estado actual)

Archivo `/etc/nginx/sites-available/itrc-web`. nginx sirve el sitio estático y proxea las rutas administrativas y de API hacia Strapi en `127.0.0.1:1337`.

```

server {
    listen 80 default_server;
    server_name _;

    root /var/www/itrc-web;
    index index.html;

    gzip on;
    gzip_types text/plain text/css application/javascript application/json
        image/svg+xml;

    location /_astro/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
    location /documentos/ {
        expires 7d;
        add_header Cache-Control "public";
    }

    # Strapi: panel admin, API, uploads, content-manager, etc.
    location ~ ^/(admin|api|uploads|content-manager|content-type-builder|
        upload|users-permissions|i18n|email|content-releases|
        review-workflows|preview)(/|$) {
        proxy_pass http://127.0.0.1:1337;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        # WebSocket (live reload, hot updates en dev)
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 90s;
    }

    # Sitio estático
    location / {
        try_files $uri $uri.html $uri/index.html =404;
    }
}

```

TLS y dominio: pendiente

Cuando se asigne un dominio público y se autorice la salida de la VPN, se añade un **server** de *listen 443* con *Let's Encrypt*, redirección 80 → 443, y los *headers* HSTS / X-Frame-Options / Referrer-Policy. Hoy todo eso está documentado en [docs/manual-operador/](#) pero sin aplicar al ser un servidor exclusivamente VPN-interno.

5 Autenticación y administración de usuarios

Dos planos de autenticación

A partir de mayo 2026 conviven dos modelos: (1) **Strapi admin** para el flujo cotidiano del webmaster (login con email + password gestionado en el propio panel de Strapi); (2) **cuentas GitHub** para los desarrolladores que tocan código y schemas. La capa de Strapi reemplaza el OAuth-con-GitHub que se planteaba en la versión 1.1 del documento.

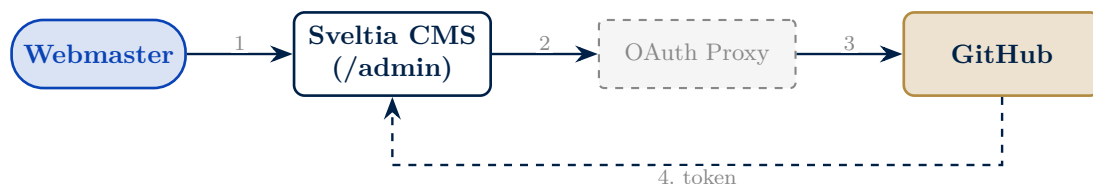
5.1 Modelo de cuentas

Editores de contenido (Strapi): cada webmaster tiene un usuario admin en Strapi (email + password). Roles del CE: Super Admin, Editor, Author. Se crean desde el propio panel (`/admin/settings/users`). Hoy en POC existe el usuario `admin@itrc.local` (Super Admin); para producción se crean cuentas individuales con role *Editor*.

Desarrolladores (GitHub): cada desarrollador cuenta con una **cuenta personal de GitHub** con permisos de **push** sobre el repositorio. No se comparten credenciales. Las razones siguen siendo:

- **Trazabilidad:** todos los commits en el historial quedan asociados a un nombre real.
- **Seguridad:** si una cuenta se compromete, se revoca su acceso individualmente sin afectar al resto.
- **Escalabilidad:** añadir o retirar personas es cuestión de un clic, sin pasar por el equipo técnico cada vez.
- **Auditoría:** cualquier auditoría posterior puede reconstruir quién hizo qué con precisión.

5.2 Flujo de autenticación (OAuth con GitHub) — *pendiente de despliegue*



1. El webmaster entra al CMS y solicita login.
2. El CMS redirige al OAuth Proxy (aloja credenciales de la app OAuth registrada en GitHub).
3. El Proxy redirige a GitHub para que el usuario autorice.
4. GitHub entrega un token al CMS via el Proxy. El CMS queda autenticado durante la sesión (típicamente 8 horas).

5.3 Alta de un webmaster nuevo

El procedimiento, a cargo del coordinador técnico:

1. El webmaster crea (o ya tiene) una cuenta personal en github.com. Se recomienda usar el correo institucional.
2. El coordinador accede al repositorio del portal en GitHub, en la sección *Settings* → *Collaborators*, y añade al nuevo webmaster con el rol **Write**.

3. GitHub envía una invitación por correo. El webmaster la acepta.
4. El webmaster clona el repositorio localmente, instala dependencias (`npm install`) y prueba `npm run dev`. Si el sitio carga, está listo para empujar cambios.
5. Cuando se reactive el CMS, también probará entrar a `<dominio>/admin/` y hacer login OAuth.

5.4 Roles sugeridos

Rol GitHub	Puede hacer	Para quién
<i>Admin</i>	Todo + gestionar colaboradores	1–2 coordinadores técnicos
<i>Maintain</i>	Push + gestionar issues	Coordinadores de contenido
<i>Write</i>	Push a main (suficiente para CMS)	Webmasters regulares
<i>Read</i>	Solo lectura	Auditoría, visores externos

5.5 Baja de un webmaster

Al retirar a una persona:

- Se remueve su acceso en *Settings* → *Collaborators* → *Remove*.
- Los commits previos quedan en el historial (no se eliminan, por auditoría).
- No se revoca la aplicación OAuth: está asociada al portal, no a la persona.

5.6 Buenas prácticas de seguridad

- Exigir **autenticación en dos pasos (2FA)** en todas las cuentas que tengan acceso al repo.
- Revisar trimestralmente la lista de colaboradores y retirar accesos inactivos.
- El *Client Secret* del OAuth App nunca debe aparecer en código ni en commits.
- Ante sospecha de cuenta comprometida: remover acceso inmediato, cambio de contraseña y rotación de 2FA por parte del usuario, y reincorporación controlada.

6 Consideraciones operativas

6.1 Manejo de binarios

Pivote de mayo 2026

La versión 1.0 de este documento describía un modelo donde los binarios vivían dentro del repositorio. Ese enfoque se descartó tras un incidente de pérdida de 3.5 GB durante un despliegue: el `rsync -delete` del workflow borró el directorio de binarios cuando el runner clonó el repo limpio (sin esos archivos). El modelo actual los mantiene fuera del repositorio.

Los archivos binarios del portal (PDFs, hojas de cálculo, imágenes pesadas, audios, videos) residen **únicamente en el servidor**, en el directorio `/var/www/itrc-web/documentos/`. No se versionan en Git. Esto implica:

- El repositorio Git contiene únicamente el código del sitio y el contenido editorial (JSON / Markdown), con un tamaño cómodo (~50 MB) que clona en segundos.
- Los binarios se suben por canal manual (`npm run deploy:binarios`, ver Escenario 3).
- El workflow del runner usa `-exclude=/documentos/` para no tocarlos durante despliegues, evitando una repetición del incidente.
- La ruta de servido en el código sigue siendo `/documentos/` a través de `DOCS_BASE_URL`.
- La fuente de verdad de los binarios es el servidor + el snapshot nocturno (ver §6.4). Cuando el operador necesita una copia, hace `rsync` desde el servidor a su máquina local.

Implicación: el CMS necesita un endpoint de uploads

Como los binarios no están en Git, el CMS visual (cuando se reactive) no puede subirlos vía `commit API`. Necesita un endpoint HTTP propio que reciba el archivo, lo deposite en `/var/www/itrc-web/documentos/` y devuelva la URL pública. Esa pieza está pendiente y depende de la decisión de CMS (§3.3).

6.2 Rollback (revertir un cambio)

Para contenido o código (en Git):

1. Identificar el commit problemático con `git log -oneline`.
2. Ejecutar `git revert <hash>` — crea un nuevo commit que deshace los cambios.
3. `git push`. El runner despliega la versión restaurada en 1–3 minutos.

Para binarios (en servidor):

1. Identificar el snapshot que tiene la versión correcta: `ls /var/backups/itrc-web/daily/`.
2. Restaurar selectivamente: `sudo cp /var/backups/itrc-web/daily/daily.N/documentos/<archivo>/var/v`
3. Si se requiere restaurar todo el directorio, usar `rsync -a` desde el snapshot (ver docs/BACKUP.md).

Nunca forzar el historial

No usar `git push -force` sobre la rama principal. El historial es auditable y debe preservarse. La forma correcta de deshacer cambios publicados es siempre con `git revert`, que deja evidencia de la corrección en el log.

6.3 Monitoreo y logs

- **Logs de nginx:** `/var/log/nginx/access.log` y `error.log`. Rotación semanal estándar.
- **Estado de despliegues:** pestaña *Actions* del repositorio en GitHub. Cada workflow se registra con SHA, autor, mensaje, duración y éxito/fallo.
- **Estado del runner:** `sudo systemctl status actions.runner.<repo>.<host>.service`. El runner debe aparecer como *active (running)* y conectado al repo.
- **Log del backup:** `/var/log/itrc-backup.log`. Buscar "FIN BACKUP" para confirmar éxito de la última corrida.
- **Disponibilidad:** monitoreo externo opcional (uptime pings) cuando exista dominio público.

6.4 Política de respaldos local

El servidor ejecuta un cron nocturno (2:00 UTC) que invoca `/usr/local/bin/itrc-backup.sh` (commiteado en el repo como `ops/server-backup.sh`). La estrategia es *Time Machine* basada en `rsync -link-dest`: cada snapshot se ve como árbol completo navegable, pero archivos sin cambios son *hardlinks* al snapshot anterior, así que el costo en disco es $\approx$ tamaño actual + suma de deltas diarios.

Tipo	Cuántos	Frecuencia
Daily	7	Cada noche, rotación FIFO
Weekly	4	Cada domingo (clonado de <code>daily.0</code> con <i>hardlinks</i>)
Monthly	6	Día 1 de cada mes (clonado de <code>daily.0</code> con <i>hardlinks</i>)
Configs (tarball)	30	Uno por día, incluye nginx, sudoers, ufw, lista de paquetes, kernel

Espacio físico estimado para volumen actual (~ 3.7 GB de webroot): 6–8 GB para todos los snapshots combinados. Con 12 GB libres en disco y umbral de alerta a 85 %, hay margen.

Limitaciones conocidas

El backup es **local al mismo servidor**. Si el servidor entero falla (disco principal corrupto, VM perdida), los respaldos se pierden con él. Mitigación pendiente: cuando se asigne una segunda máquina institucional, configurar *push* periódico de los snapshots y los *tarballs* de configs a esa máquina (o a un repo Git privado para los configs, que son livianos).
Tampoco hay **notificación de fallo** hoy: el cron tiene `MAILTO=daniel@digitalia.gov.co` pero `postfix` no está instalado. Mientras: revisar `/var/log/itrc-backup.log` periódicamente.

Detalle completo de la política, estructura en disco y procedimientos de restauración en `docs/BACKUP.md`.

6.5 Respaldo del repositorio (Git)

Para el lado Git, el modelo provee respaldo implícito:

- El repositorio en GitHub es un backup por sí mismo (replicado en infraestructura GitHub).
- Cada webmaster con su clon local tiene otra copia completa.
- El historial Git permite restaurar cualquier versión anterior por fecha o commit.

Referencias

Documentos internos del repositorio (fuente de verdad operativa):

- docs/DEPLOY.md — Detalle de los tres flujos de despliegue (auto, fallback, binarios) y diagnóstico de fallos.
- docs/BACKUP.md — Política de respaldos, estructura en disco y procedimientos de restauración.
- docs/manual-operador/ — Manual del operador (instalación, configuración, mantenimiento del servidor).
- ops/deploy.sh, ops/deploy-binarios.sh, ops/server-backup.sh — Scripts ejecutables.
- .github/workflows/deploy.yml — Workflow del runner self-hosted.
- REFERENCIAS-FALTANTES.md — Auditoría de enlaces y omisiones detectadas durante la migración.

Documentación externa:

- Astro: <https://astro.build>
- Strapi v5 (Community Edition): <https://docs.strapi.io>
- Postgres 16: <https://www.postgresql.org/docs/16/>
- Docker Compose: <https://docs.docker.com/compose/>
- GitHub Actions self-hosted runners: <https://docs.github.com/actions/hosting-your-own-runners>
- GitHub repository_dispatch: <https://docs.github.com/en/rest/repos/repos#create-a-repository-dispatch-event>
- nginx documentation: <https://nginx.org/en/docs/>
- Let's Encrypt / Certbot: <https://certbot.eff.org>
- rsync *link-dest* (estrategia Time Machine): <https://rsync.samba.org/documentation.html>